Intelligent Security Camera Placement & Interference Management

Exam explanation guide - DAA project (Daa-Fa2)

Backtracking for placement + conflict graph + Graph Coloring (also backtracking) for channel assignment. This guide is written so you can explain the project from scratch, walk through the code, run a live demo, and answer likely viva questions.

Section	What it covers
1	The problem in one minute (what to say first)
2	Big picture: the 3 stages
3	Stage 1: Backtracking placement (rules, algorithm, pseudocode)
4	Stage 2: Conflict graph
5	Stage 3: Graph coloring with backtracking
6	Worked example from the real app (numbers from the code)
7	Complexity analysis
8	Code walkthrough: file by file
9	Optimizations I made
10	Live demo script
11	Limitations (be honest about these)
12	Viva questions with answers

1. The problem in one minute

Scenario. A building floor is a grid of cells. We must place K security cameras on it. Some cells are blocked (walls, pillars). Cameras must not be placed in conflicting positions (same row / column / diagonal, or too close together). Wireless cameras that are near each other interfere if they use the same communication channel, so after placing them we must give every camera a channel such that no two interfering cameras share one.

Why it is a DAA problem. Both halves are constraint-satisfaction problems with no known polynomial algorithm in general, so we use **backtracking**: build the answer one choice at a time, check constraints immediately, and undo (backtrack) a choice as soon as it leads to a dead end.

- Part 1 is an **N-Queens style** placement problem (that is the textbook backtracking problem).
- Part 2 is the **m-coloring problem** on a graph (another textbook backtracking problem; it is NP-complete for m >= 3).
- The app is an **interactive simulator**: every decision the algorithms make is recorded as a step and replayed visually, so you can watch backtracking happen.

One-line summary to open with: "My project places cameras on a grid using backtracking, builds an interference graph between them, and then colors that graph with backtracking to assign non-conflicting channels. A web app visualizes every step."

2. Big picture: the three stages

```
CONFIG (rows, cols, cameras K, channels m, interference range,
        min separation, row/col/diag toggles, blocked cells)
       |
       v
STAGE 1  Backtracking placement  -> list of K camera cells   (or "no placement")
       |
       v
STAGE 2  Conflict graph          -> edge if distance <= range
       |
       v
STAGE 3  Graph coloring (m ch.)  -> channel per camera       (or "not colorable")
       |
       v
RESULTS  table, chart, JSON export
```

The three stages run once, up front, and produce one long list of **steps** (try / reject / place / backtrack / edge / assign / conflict ...). The UI is just a player over that list: Start, Pause, Next, Prev, Skip phase, Jump to result, speed slider, scrubbable timeline. This separation (pure algorithms vs. UI) is a deliberate design choice.

3. Stage 1: Backtracking camera placement

3.1 Constraints (fixed and documented)

#	Constraint	Toggle?
1	Cell is not blocked	always
2	No two cameras in the same row	yes
3	No two cameras in the same column	yes

#	Constraint	Toggle?
4	No two cameras on the same diagonal: |r1-r2| = |c1-c2|	yes
5	Chebyshev distance max(|dr|,|dc|) >= minimum separation	always (1 = may touch)

With rows+columns+diagonals on and no separation rule this is exactly **N-Queens**, which is why I call the model "inspired by N-Queens".

3.2 How the search works

- Camera k is tried on cells **strictly after** camera k-1 in row-major order. This means a set of cells is never explored in multiple orders (no permutations of the same set), so cells never repeat.
- For each candidate cell: check constraints against every camera already placed. If a rule fails, **reject** and try the next cell. If all pass, **place** the camera and recurse for camera k+1.
- If the recursion for k+1 fails, **backtrack**: remove camera k and continue with its next candidate cell.
- **Pruning:** if the number of cells left is less than the cameras still to place, stop that loop (no way to finish).
- **Safety cap:** 60,000 steps. An impossible big scenario cannot freeze the browser; the log then says the search was aborted.

3.3 Pseudocode

```
solve(start):
    if placed.size == K: return true
    remaining = K - placed.size
    for idx = start .. totalCells - remaining:        # prune
        cell = (idx / cols, idx % cols)
        if violates(cell, placed): record reject; continue
        placed.push(cell); record place
        if solve(idx + 1): return true
        placed.pop();     record backtrack
    return false
```

Backtracking in one sentence: a depth-first search of the decision tree that abandons a partial solution the moment a constraint is violated, instead of generating every full arrangement and filtering afterwards.

4. Stage 2: Conflict (interference) graph

- **Vertices** = the placed cameras C1..CK.
- **Edge** between two cameras iff their **Euclidean** distance <= interference range (in cells).
- Built by checking every pair: $O(V^2)$ time. Stored as an adjacency list: O(V+E) space.
- A tiny 1e-9 tolerance is added so values like 2.0 vs range 2 do not fail from floating-point error.

Note the difference between the two distance rules: placement uses Chebyshev distance (a hard constraint on where cameras may go), while interference uses Euclidean distance (a soft physical idea: radio range). They are intentionally different.

5. Stage 3: Graph coloring with backtracking

- Colors = communication channels 1..m. Goal: give every vertex a channel so that **adjacent vertices differ** (a proper coloring).
- **Vertex order:** descending degree (most-constrained first). It is a standard heuristic: hard vertices fail early, so dead ends are found sooner.
- **Channel order:** try 1, 2, ... m. Before assigning, look at neighbours; if a neighbour already uses that channel record a **conflict** and try the next.
- **Symmetry breaking (my optimization):** never try a channel greater than (highest channel used so far + 1). Channels are interchangeable, so skipping the rest loses no solutions but removes duplicate work.
- If no proper coloring exists the app reports it and suggests adding channels or reducing the range.

```
color(i, used):
    if i == n: return true
    v = order[i]
    for ch = 1 .. min(m, used + 1):              # symmetry breaking
        if some neighbour u of v has color[u] == ch: record conflict; continue
        color[v] = ch; record assign
        if color(i + 1, max(used, ch)): return true
        color[v] = none; record backtrack
    return false
```

6. Worked example (real numbers from the program)

6.1 Default "Classroom demo": 8x8 grid, 6 cameras, 3 channels, range 3, min separation 2

Blocked cells: (1,1), (2,2), (3,3). Coordinates are (row,col), 1-indexed as shown in the app.

Result	Value
Cameras found	C1(1,2) C2(2,4) C3(3,1) C4(4,3) C5(5,8) C6(6,6)
Placement work	73 positions tried, 66 conflicts, 1 backtrack
Conflict edges (distance)	C1-C2 2.24, C1-C3 2.24, C2-C4 2.24, C3-C4 2.24, C5-C6 2.24 (5 edges)
Channel assignment	C1=1, C2=2, C3=2, C4=1, C5=1, C6=2 (only 2 of 3 channels needed)
Coloring work	9 tried, 3 conflicts, 0 backtracks
Total steps replayed	173

Check by hand: every edge joins different channels (C1-C2: 1/2, C1-C3: 1/2, C2-C4: 2/1, C3-C4: 2/1, C5-C6: 1/2). The graph is bipartite, so 2 colors are enough.

6.2 All four built-in scenarios

Scenario	Outcome	Key numbers
Classroom demo	Success	173 steps, 1 backtrack, 5 edges
Heavy backtracking (6x6, 6 cams, range 3.5)	Success, lots of backtracking	1334 positions tried, 201 backtracks, 7 edges, 2897 steps
Needs more channels (range 5, only 2 channels)	Placement OK, coloring FAILS	10 edges form triangles (C1,C2,C3...) so 2 colors are impossible

Scenario	Outcome	Key numbers
No placement exists (4x4, 5 cameras, rows+cols unique)	Placement FAILS	Pigeonhole: 5 cameras, only 4 rows. 225 tried, 54 backtracks before search exhausts

7. Complexity analysis

Stage	Time	Space	Explanation
Placement	Exponential worst case (bounded by C(RC, K) combinations; each check O(K))	O(K) recursion stack	Pruning and early constraint checks cut the tree massively; worst case still exponential.
Graph construction	$O(V^2)$	O(V+E)	All pairs of cameras.
Coloring	$O(m^V)$ worst case	O(V)	At most m choices at each of V levels. Symmetry breaking reduces it to roughly the number of set partitions, still exponential.
Replay / UI	O(1) per step	O(steps x K)	Steps are precomputed, each a snapshot.

Why not polynomial? N-Queens-type placement and m-coloring (m >= 3) are NP-hard / NP-complete in general; no known polynomial algorithm. Backtracking is an exact method with good practical pruning, not a polynomial one.

8. Code walkthrough: file by file

Tech: React 19 + TypeScript + Vite, Zustand for state, Tailwind for styling, Framer Motion for animation, React Flow for the draggable graph, Recharts for the results chart, Vitest for tests.

File	Role
src/algorithms/backtracking.ts	checkPlacement() = the 5 constraints; solvePlacement() = recursive backtracking that logs steps. Pure TypeScript, no React.
src/algorithms/conflictGraph.ts	buildConflictGraph() (pairwise distance test), adjacencyList(), graphSteps() (one step per edge).
src/algorithms/graphColoring.ts	colorGraph(): backtracking m-coloring with degree ordering and symmetry breaking.
src/types/simulation.ts	Config, Cell, Edge, step types (PlacementStep, GraphStep, ColoringStep), Stats.
src/data/defaultScenario.ts	Default config and the 4 presets.
src/store/simulationStore.ts	Zustand store. compile(config) runs all 3 stages and concatenates the steps. Player actions: start/play/pause/next/prev/seek/skipPhase/finish/reset. deriveView() turns "current step index" into what the UI should draw.

File	Role
src/components/*	BuildingGrid + CameraCell (floor plan), ConflictGraph (React Flow), ColoringPanel, AlgorithmStats (live counters), StepLog (narration), ResultsPanel (table, chart, JSON export), ConfigurationPanel, AlgorithmControls, PhaseStepper.
src/algorithms/algorithms.test.ts	11 unit tests: valid placement, no-solution case, blocked cells, diagonal rule, backtracking occurs, edges by range, proper coloring, triangle with 2 colors fails, symmetry breaking, snapshots, compile stats.

Key design idea: record, then replay

The algorithms do not touch the UI. They run to completion and push a record of every decision into an array. The UI keeps just an index into that array; "Next" is index+1, "Prev" is index-1, "Skip phase" jumps to the last step with the same phase. This is why stepping backwards and a scrubbable timeline are trivial, and why the algorithms are easy to unit test.

9. Optimizations I made

Change	Before	After / benefit
Blocked-cell lookup	Linear scan of blocked list for every candidate	Set lookup, O(1)
Placement step snapshots	Each step copied the whole camera list	Steps share one snapshot until a camera is placed/removed: less memory
Coloring symmetry breaking	Retried equivalent channel permutations	Never tries channel > used+1: impossible instances (e.g. 6 mutually interfering cameras, 5 channels) fail in under 100 steps instead of hitting the 60,000 cap
Final placement stats	Panels filtered up to 60,000 steps on every playback tick	Computed once in compile()
Store subscriptions	Components subscribed to the whole store, re-rendering every tick	Components select only the slices they use
Grid rendering	Per-cell scans for blocked/camera	Memoized Set/Map lookups
Input clamping	Interference range and min separation unbounded	Clamped like the other settings; truncation of coloring search is now tracked

Deliberately **not** done: an instant "more cameras than rows" feasibility check, because it would skip the backtracking demonstration for the "No placement exists" scenario, which is the educational point.

10. Live demo script (about 3 minutes)

- Run `npm install` then `npm run dev`, open http://localhost:5173.
- **1. Show the grid** with the default scenario. Click a cell to block it (only possible before starting). Explain the toggles.

- **2. Press Start.** Point out: dashed cell = candidate being tried, orange shake = conflict (the reason is in the narration log), green camera = placed, "backtracked" cell = removed after a dead end.
- **3. Phase 2.** Edges appear on the floor plan and in the draggable conflict graph. Hover/drag nodes; edge labels are distances.
- **4. Phase 3.** Cameras get channel colors one at a time; the dashed ring shows the interference radius of the camera being colored; orange edge = clash with a neighbour.
- **5. Results.** Show the table, bar chart (backtracking vs coloring effort) and Export JSON.
- **6. Presets.** Load "Heavy backtracking" to show 200+ backtracks; "Needs more channels" to show a coloring failure; "No placement exists" to show a placement failure and explain the pigeonhole reason.
- **7. Speed slider / timeline:** scrub backwards to re-show any decision.

Tests: `npm test` (11 passing). Production build: `npm run build`.

11. Limitations (say these before the examiner asks)

- It is an **educational model**, not RF engineering or real CCTV planning: no field of view, mounting, walls blocking signal, bandwidth or propagation.
- The placement problem treats cameras as points and uses N-Queens-style rules that are an abstraction of "spread cameras out".
- Backtracking is exponential: big or impossible inputs are cut off at 60,000 steps (the app tells you).
- The first solution found is returned, not an optimal one (e.g. minimum number of channels is not minimized; the app just checks if m channels suffice).
- Greedy/DSATUR or ILP could be faster for large graphs; backtracking was chosen because the course topic is backtracking and because it is exact and visualizable.
- All algorithms run on the browser main thread; a Web Worker would be the next step for big grids.

12. Viva questions with answers

Q. What is backtracking?

A depth-first search over partial solutions. We extend a partial solution one choice at a time, check constraints immediately, and if a choice leads to a dead end we undo it and try the next. It prunes entire subtrees, unlike brute force which builds every full candidate first.

Q. How is this different from brute force?

Brute force would generate all C(RC, K) placements and test each. Backtracking rejects a partial placement the moment it violates a rule, so the huge majority of the tree is never visited.

Q. Which classic problems does the project use?

N-Queens for placement (row/column/diagonal rules) and m-coloring of a graph for channel assignment.

Q. Why do you try cells in increasing row-major order?

To avoid exploring the same set of cells in different orders. Cameras are indistinguishable for placement, so combinations, not permutations, are enough. It also guarantees no duplicate cells.

Q. What is the pruning in the placement loop?
If the cells remaining are fewer than the cameras still to place, no completion exists, so the loop stops. Constraint checks also prune immediately on each candidate.

Q. Why is the interference graph built after placement, not during?
Interference does not restrict where cameras can stand; it only affects their channels. So the stages are independent: place first, then model conflicts, then color.

Q. Why different distances (Chebyshev vs Euclidean)?
Chebyshev is the rule for physical spacing on a grid (king-move distance). Euclidean models radio range as a circle. They answer different questions.

Q. Why order vertices by degree?
Highest-degree vertices have the most constraints; coloring them first makes failures appear near the root of the search tree, which prunes more.

Q. What is symmetry breaking and is it correct?
Channel names are interchangeable (swapping 1 and 2 everywhere gives another valid coloring). Restricting each vertex to at most (max used)+1 means we only explore one representative of each such family. It never removes all colorings: any coloring can be relabelled so colors first appear in increasing order.

Q. What is the time complexity?
Placement: exponential worst case. Graph build: O(V^2). Coloring: O(m^V) worst case. Space: O(K) for placement recursion, O(V+E) for the graph, O(V) for coloring.

Q. Is graph coloring NP-complete?
Deciding whether a graph is m-colorable is NP-complete for m >= 3 (for m = 2 it is polynomial: it is a bipartiteness check). So an exponential exact algorithm is expected.

Q. What does it mean when "Needs more channels" fails?
The conflict graph contains a structure (e.g. a triangle, which needs 3 colors) that cannot be colored with 2 channels. The fix is more channels or a smaller interference range.

Q. Why does "No placement exists" fail for 4x4 with 5 cameras?
With one camera per row (and per column) at most 4 cameras fit on a 4x4 grid: pigeonhole. The search still explores the tree to prove it (225 tries, 54 backtracks).

Q. How does the visualization work without slowing the algorithm?
The algorithm runs once and logs every decision into a step array. The UI only moves an index over that array, so replay, back-step and scrubbing are cheap.

Q. How do you stop infinite or huge searches?
A hard cap of 60,000 steps per phase. When reached, the search aborts and the log says so.

Q. How did you test correctness?
Vitest unit tests: placement validity of every camera against earlier ones, blocked cells respected, diagonal rule toggle, no-solution detection, graph edges only within range, proper coloring (no edge has equal endpoints), triangle with 2 colors fails, and the optimization behaviours.

Q. What would you improve next?
Web Worker for the searches, forward checking / constraint propagation, DSATUR ordering, scenario comparison, and a more realistic interference model (walls, field of view).

Q. Which optimization gave the biggest effect?

Symmetry breaking in coloring: impossible instances that previously ran to the step cap now terminate in a handful of steps. The UI fixes (precomputed stats, store selectors) removed O(steps) work per playback tick.